

AI-ASSIGNMENT-9.4

2303A52249

A.VAMSHI KRISHNA

TASK-1:

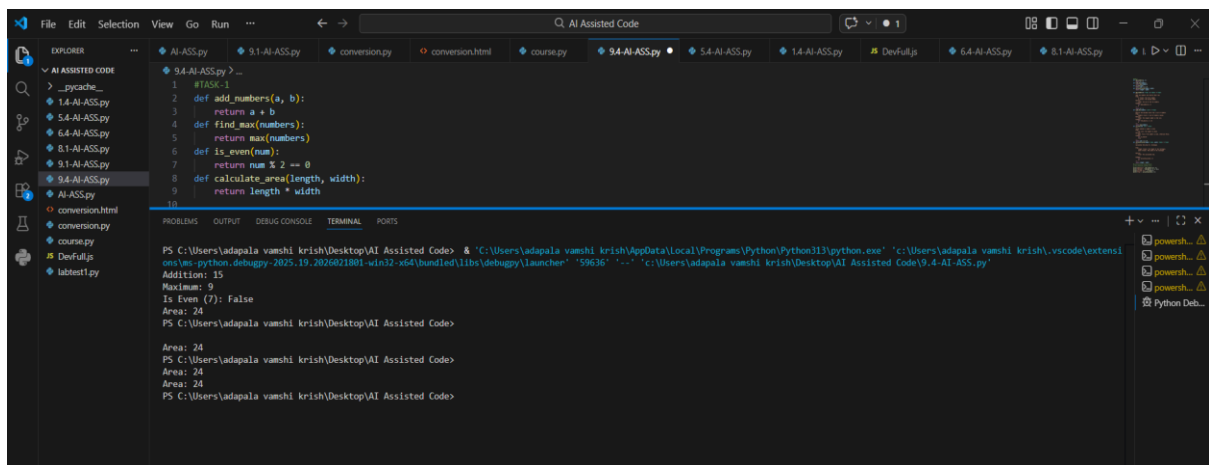
Scenario: Auto-Generating Function Documentation. You have joined a software development team where several utility functions are already implemented in a shared Python codebase. However, the functions do not contain proper documentation or docstrings. New team members are facing difficulty understanding.

Prompt: Generate Google-style docstrings for the following Python functions and Include:

- Short description
- Args section with data types
- Returns section
- Example usage

Code:

Step 1: Original Script (Without Documentation)

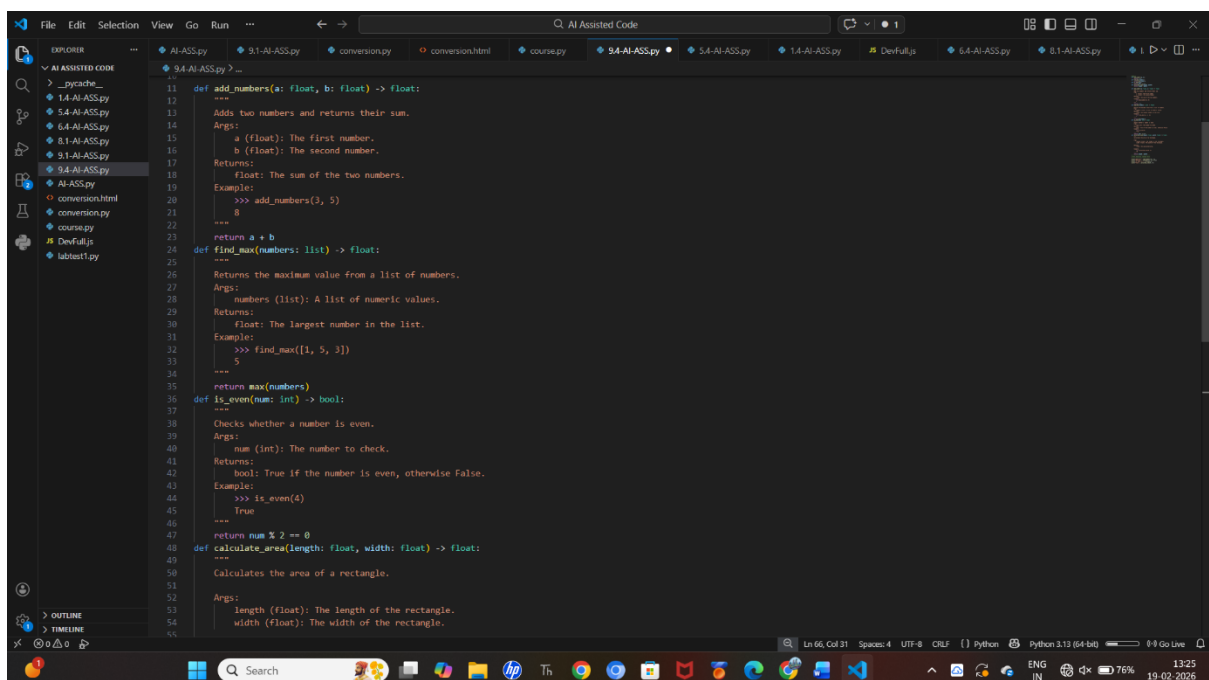


The screenshot shows a VS Code editor with a Python file named '9.4-AI-ASS.py'. The code contains four functions: `add_numbers(a, b)`, `find_max(numbers)`, `is_even(num)`, and `calculate_area(length, width)`. The terminal output shows the execution of these functions, displaying the results of the calculations.

```
1 #TASK-1
2 def add_numbers(a, b):
3     return a + b
4 def find_max(numbers):
5     return max(numbers)
6 def is_even(num):
7     return num % 2 == 0
8 def calculate_area(length, width):
9     return length * width
10
```

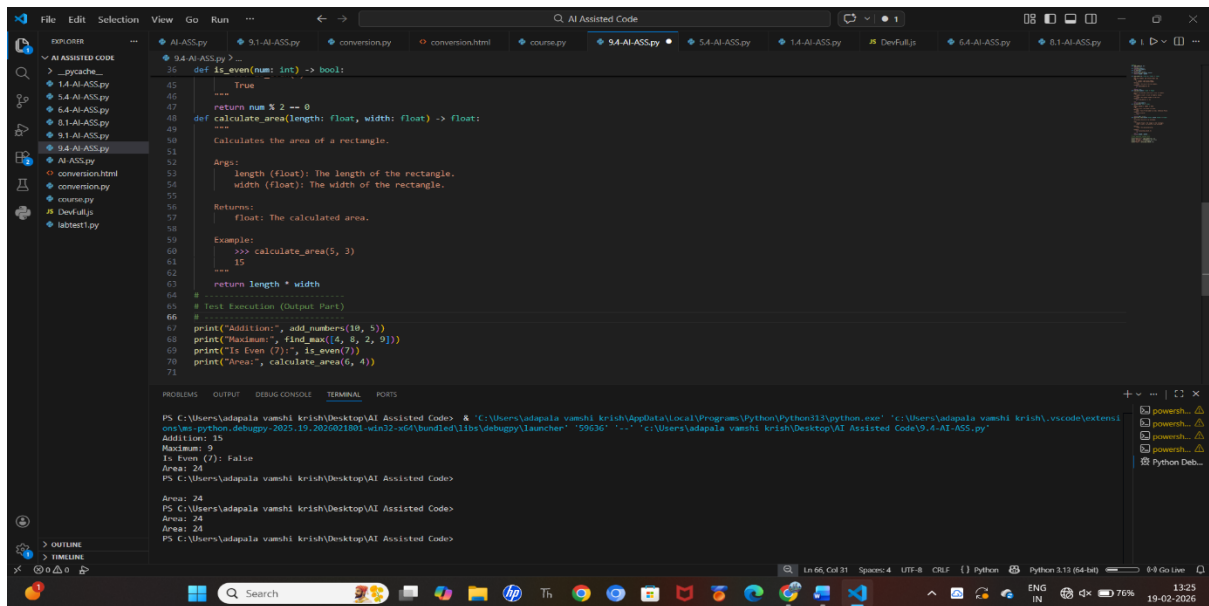
```
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code> & "C:\Users\adapala vamshi krish\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\adapala vamshi krish\vscode\extensions\ms-python.python.debugger-2025.19.2026021801-win32-x64\bundled\libs\debugpy\launcher" "59636" "--" "c:\Users\adapala vamshi krish\Desktop\AI Assisted Code\9.4-AI-ASS.py"
Addition: 15
Maximum: 9
Is Even (7): False
Area: 24
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
Area: 24
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
Area: 24
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
Area: 24
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>
```

Step 2: Script With Google-Style Docstrings (AI Generated)



The screenshot shows the same VS Code editor with the updated Python file '9.4-AI-ASS.py'. The code now includes Google-style docstrings for each function, providing a short description, arguments, and return values. The terminal output is the same as in Step 1.

```
11 def add_numbers(a: float, b: float) -> float:
12     """
13     Adds two numbers and returns their sum.
14     Args:
15         a (float): The first number.
16         b (float): The second number.
17     Returns:
18         float: The sum of the two numbers.
19     Example:
20     >>> add_numbers(3, 5)
21     8
22     """
23     return a + b
24 def find_max(numbers: list) -> float:
25     """
26     Returns the maximum value from a list of numbers.
27     Args:
28         numbers (list): A list of numeric values.
29     Returns:
30         float: The largest number in the list.
31     Example:
32     >>> find_max([1, 5, 3])
33     5
34     """
35     return max(numbers)
36 def is_even(num: int) -> bool:
37     """
38     Checks whether a number is even.
39     Args:
40         num (int): The number to check.
41     Returns:
42         bool: True if the number is even, otherwise False.
43     Example:
44     >>> is_even(4)
45     True
46     """
47     return num % 2 == 0
48 def calculate_area(length: float, width: float) -> float:
49     """
50     Calculates the area of a rectangle.
51     Args:
52         length (float): The length of the rectangle.
53         width (float): The width of the rectangle.
54     """
```



Observation:

After generating documentation using different prompting styles, the following observations were made:

◆ Zero-Shot Prompt Results

Documentation was generated correctly.
Basic structure was present.
Some examples were missing or unclear.
Formatting was acceptable but not highly professional.

◆ Context-Based Prompt Results

Output was more structured and professional.
All required sections were properly included.
Clear type annotations were added.
Example usage was consistently provided.
Overall readability improved significantly.

TASK-2:

Scenario: Enhancing Readability Through AI-Generated Inline Comments. A Python program in the shared codebase contains complex logic involving loops, conditional statements, and algorithmic implementations such as sorting and Fibonacci sequence generation.

Prompt:

Zero-Shot

Insert meaningful inline comments into the following Python code.
Comment only complex or non-obvious logic.
Do not comment simple syntax or obvious lines.
Explain why the logic exists, not what Python does.

Context-Based

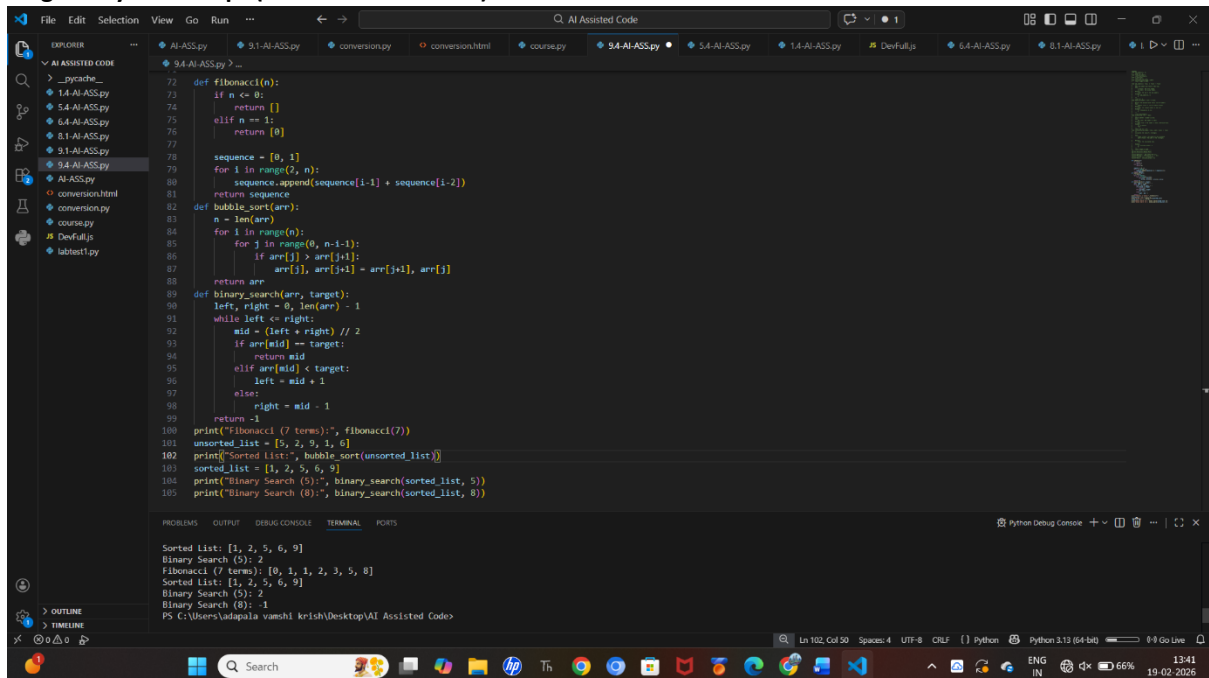
You are a senior Python developer reviewing code for maintainability. Add concise inline comments to the following script.

Rules:

- Comment only non-obvious logic
- Explain reasoning behind algorithm steps
- Avoid commenting trivial syntax
- Keep comments professional and minimal

Code:

Original Python Script (Without Comments)



The screenshot shows a VS Code editor with a Python script. The script contains three functions: `fibonacci`, `bubble_sort`, and `binary_search`. The `fibonacci` function calculates the 7th Fibonacci number. The `bubble_sort` function sorts a list of numbers. The `binary_search` function searches for a target in a sorted list. The script also includes test calls for each function. The terminal output shows the results of these tests.

```
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    sequence = [0, 1]
    for i in range(2, n):
        sequence.append(sequence[i-1] + sequence[i-2])
    return sequence

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

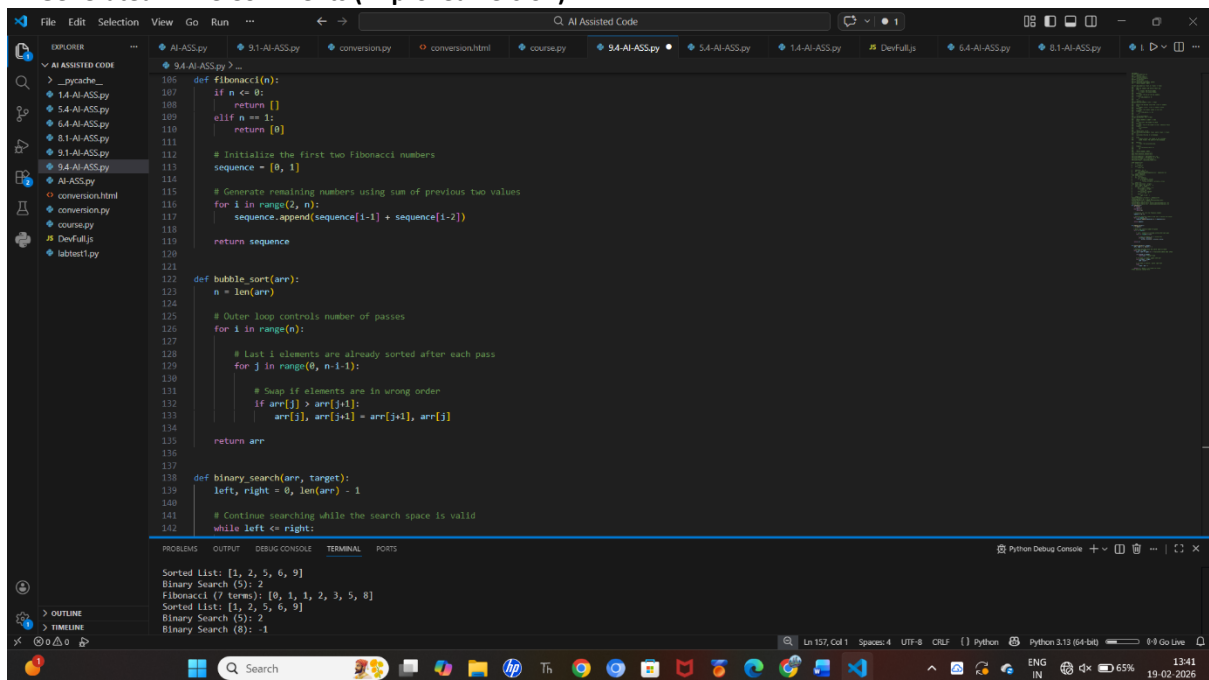
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

print("Fibonacci (7 terms):", fibonacci(7))
unsorted_list = [5, 2, 9, 1, 6]
print("Sorted List:", bubble_sort(unsorted_list))
sorted_list = [1, 2, 5, 6, 9]
print("Binary Search (5):", binary_search(sorted_list, 5))
print("Binary Search (8):", binary_search(sorted_list, 8))
```

Terminal Output:

```
Sorted List: [1, 2, 5, 6, 9]
Binary Search (5): 2
Fibonacci (7 terms): [0, 1, 1, 2, 3, 5, 8]
Sorted List: [1, 2, 5, 6, 9]
Binary Search (5): 2
Binary Search (8): -1
```

AI-Generated Inline Comments (Improved Version):



The screenshot shows the same VS Code editor with the Python script, but now with AI-generated inline comments. The comments explain the purpose of each function and the logic within the loops and conditions. The terminal output is the same as in the previous screenshot.

```
def fibonacci(n):
    if n <= 0:
        return []
    elif n == 1:
        return [0]
    # Initialize the first two Fibonacci numbers
    sequence = [0, 1]
    # Generate remaining numbers using sum of previous two values
    for i in range(2, n):
        sequence.append(sequence[i-1] + sequence[i-2])
    return sequence

def bubble_sort(arr):
    n = len(arr)
    # Outer loop controls number of passes
    for i in range(n):
        # Last i elements are already sorted after each pass
        for j in range(0, n-i-1):
            # Swap if elements are in wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    # Continue searching while the search space is valid
    while left <= right:
```

Terminal Output:

```
Sorted List: [1, 2, 5, 6, 9]
Binary Search (5): 2
Fibonacci (7 terms): [0, 1, 1, 2, 3, 5, 8]
Sorted List: [1, 2, 5, 6, 9]
Binary Search (5): 2
Binary Search (8): -1
```

Observation:

◆ Before AI Assistance

Code worked correctly

Difficult to understand algorithm flow

No explanation of why certain loops or conditions exist

Hard for new developers to follow

◆ After AI-Generated Comments

- Algorithms are easier to understand
- Comments explain reasoning behind:
 - Fibonacci sequence generation
 - Bubble sort optimization logic
 - Binary search boundary adjustments
- Code readability improved significantly
- No unnecessary clutter added

TASK-3

Scenario: Generating Module-Level Documentation for a Python Package Your development team is preparing a Python module to be shared internally within the organization and later uploaded to a repository.

Prompt:

Context-Based & Python Module Code (With AI-Generated Module-Level Docstring)

You are a senior Python developer preparing a module for a professional repository. Generate a detailed module-level docstring for the following Python file. The docstring should include:

- Purpose of the module
- Required libraries or dependencies
- Brief description of key functions and classes
- Short example demonstrating how to use the module
- Professional and clear tone

Code:

```

157 import statistics
158
159 def calculate_average(marks):
160     """Returns the average of a list of marks."""
161     return statistics.mean(marks)
162
163 def find_top_student(students):
164     """
165     Returns the student dictionary with the highest average marks.
166     """
167     return max(students, key=lambda s: calculate_average(s["marks"]))
168
169 class Student:
170     """Represents a student with name and marks."""
171
172     def __init__(self, name, marks):
173         self.name = name
174         self.marks = marks
175
176     def get_average(self):
177         return calculate_average(self.marks)
178
179 if __name__ == "__main__":
180     student1 = {"name": "Kavi", "marks": [75, 80, 85]}
181     student2 = {"name": "Anita", "marks": [80, 92, 79]}
182
183     print("Average Marks:", calculate_average([80, 90, 85]))
184     print("Top Student:", find_top_student([student1, student2])["name"])
185
186     s = Student("Kiran", [70, 75, 80])
187     print("Student Average:", s.get_average())
188
189

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

e' 'c:\Users\adapala vamshi krish\vscode\extensions\ms-python.debugpy-2025.19.2026021801-win32-x64\bundled\libs\debugpy\launcher' '61490' '-' 'c:\Users\adapala vamshi krish\Desktop\AI Assisted Code\9.4-AI-ASS.py'
Average Marks: 85
Top Student: Anita
Student Average: 75
PS C:\Users\adapala vamshi krish\Desktop\AI Assisted Code>

```

Observation:

- ◆ Before AI Assistance

No clear overview at the top of the file

Users must read entire code to understand purpose

No usage example provided

Not suitable for repository sharing

- ◆ After AI-Generated Module Docstring

Clear explanation of module purpose

Dependencies clearly listed

Key functions and class described

Example usage provided

Professional structure suitable for real-world projects

TASK-4

Scenario: Converting Developer Comments into Structured Docstrings In a legacy Python project, developers have written long explanatory inline comments inside function bodies instead of using proper docstrings.

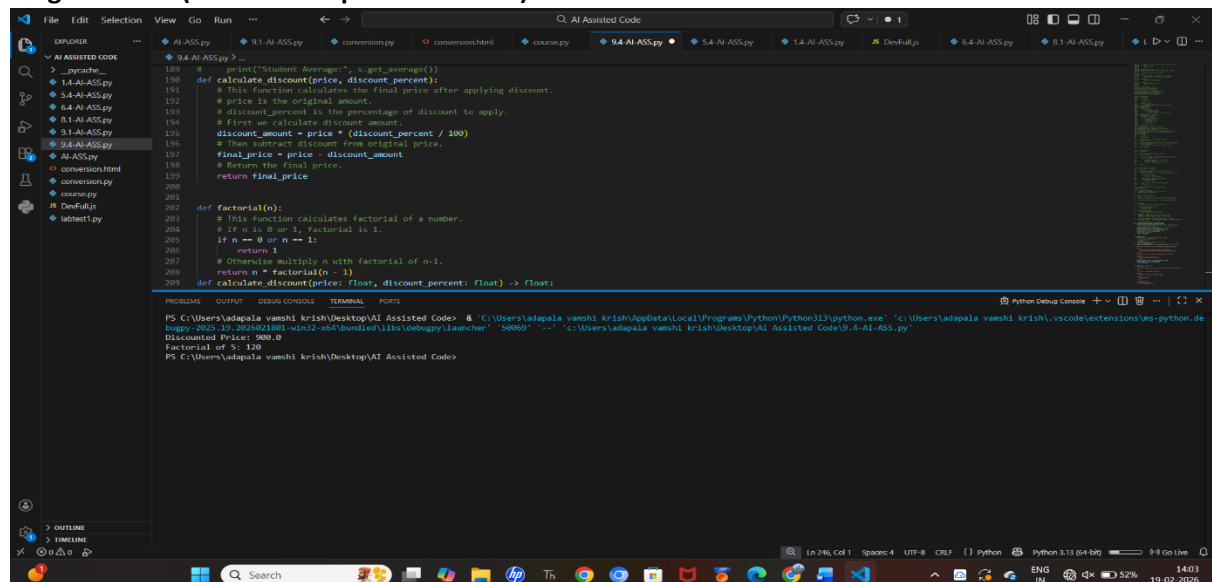
Prompt:

You are a senior Python developer. Convert the following functions that contain detailed inline comments into structured Google-style docstrings. Requirements:

- Preserve original meaning
- Include description, Args, Returns, and Example
- Remove redundant inline comments
- Maintain professional tone

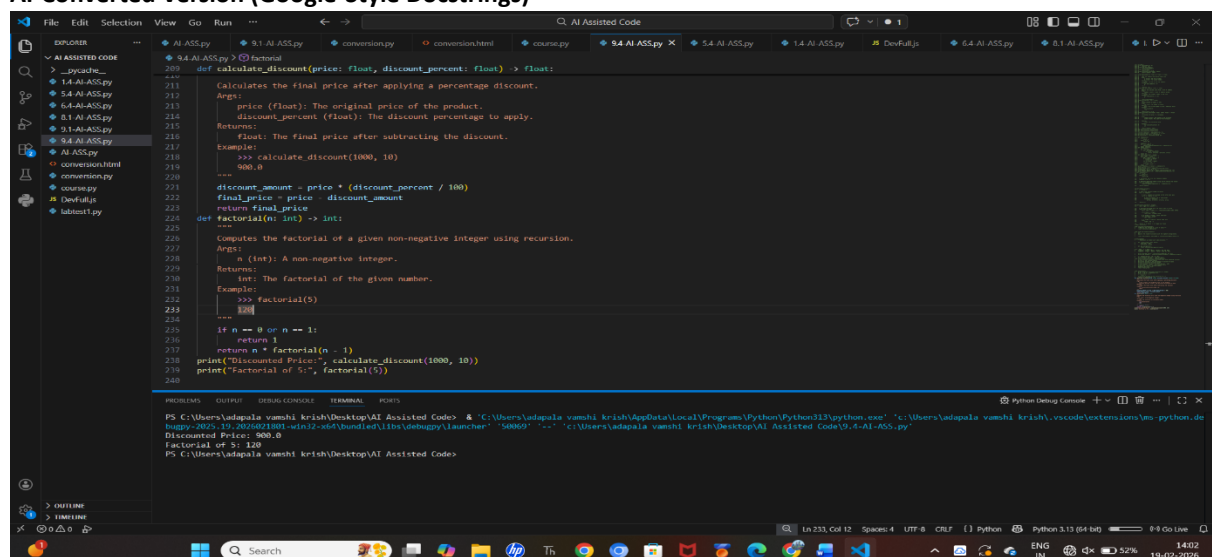
Code:

Original Code (With Developer Comments)



The screenshot shows a VS Code editor with a file explorer on the left containing files like `9.4-AI-ASS.py`, `conversion.py`, and `conversion.html`. The main editor displays the content of `9.4-AI-ASS.py`, which includes two functions: `calculate_discount` and `factorial`. The `calculate_discount` function has several lines of inline comments explaining its logic, such as "# This function calculates the final price after applying discount." and "# price is the original amount." The `factorial` function also has inline comments like "# This function calculates factorial of a number." and "# If n is 0 or 1, factorial is 1." The bottom of the window shows a terminal with the command prompt and the output of running the script, which displays the discounted price and the factorial of 5.

AI-Converted Version (Google-Style Docstrings)



The screenshot shows the same VS Code editor with the `9.4-AI-ASS.py` file, but now the code has been converted to use structured Google-style docstrings. The `calculate_discount` function's docstring includes a description, an 'Args' section for `price` and `discount_percent`, a 'Returns' section, and an 'Example' section. The `factorial` function's docstring also includes a description, an 'Args' section for `n`, a 'Returns' section, and an 'Example' section. The terminal at the bottom shows the same command prompt and output as the previous screenshot, confirming that the code still runs correctly after the conversion.

Observation:

Before Conversion

- Documentation scattered inside function body
- Hard to quickly identify function purpose
- Code cluttered with excessive comments
- No standardized format

After AI Conversion

- Clear Google-style docstrings added
- Inline clutter removed
- Function bodies are clean and readable
- Documentation standardized across module
- More professional and maintainable

TASK-5

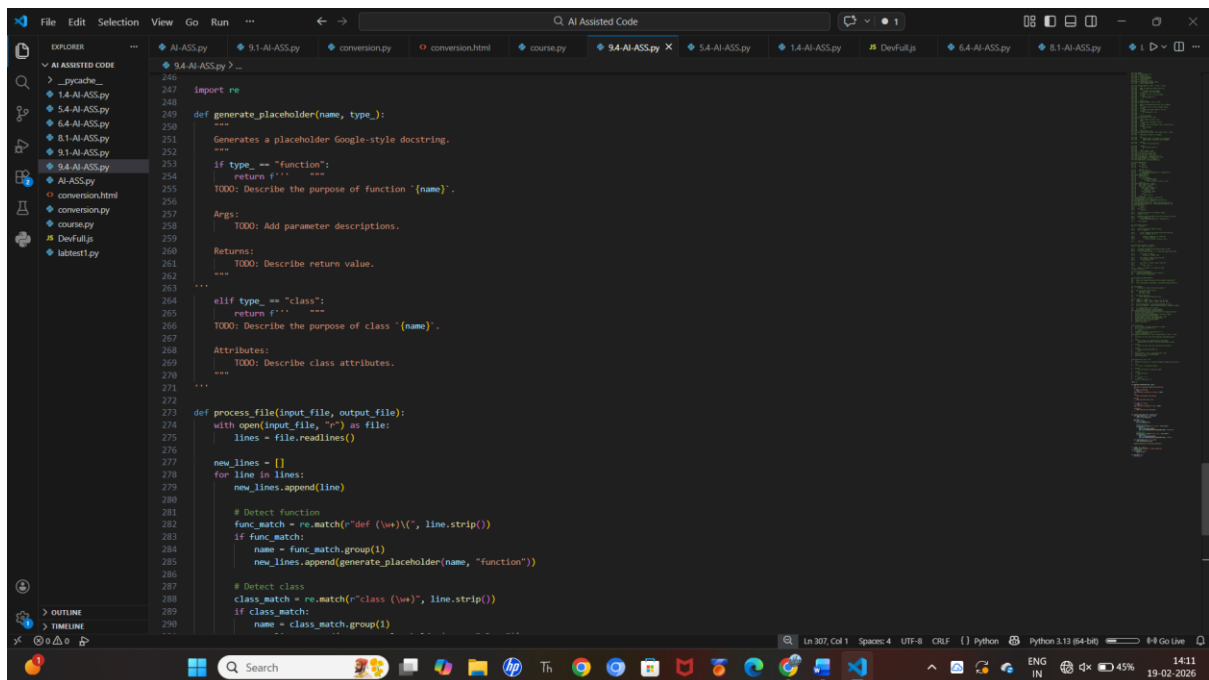
Scenario: Building a Mini Automatic Documentation Generator The development team wants a simple internal tool to help developers quickly scaffold documentation for new Python files.

Prompt: Create a Python script that:

-> Reads a given .py file -> Detects functions and classes -> Inserts placeholder Google-style docstrings

-> Writes the updated content to a new file Keep the implementation simple and readable.

Code:



```
import re

def generate_placeholder(name, type_):
    """
    Generates a placeholder Google-style docstring.
    """
    if type_ == "function":
        return f"""
    """
    elif type_ == "class":
        return f"""
    """

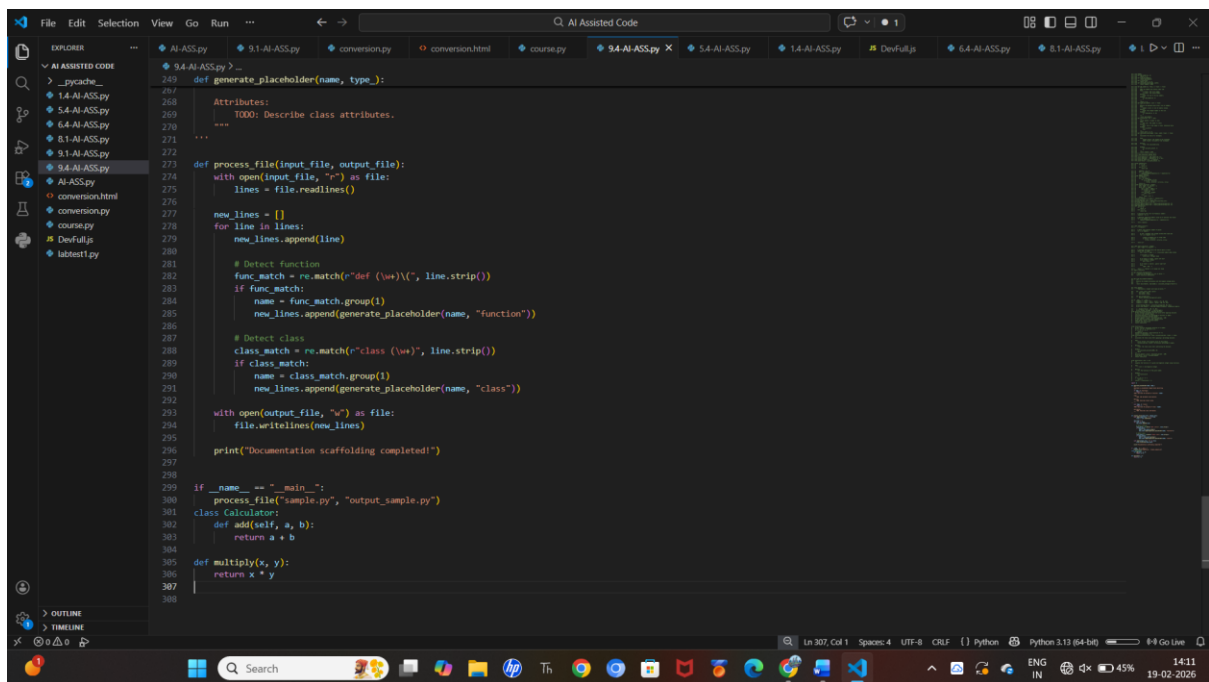
def process_file(input_file, output_file):
    with open(input_file, "r") as file:
        lines = file.readlines()

    new_lines = []
    for line in lines:
        new_lines.append(line)

        # Detect function
        func_match = re.match(r"def (.*?)\(", line.strip())
        if func_match:
            name = func_match.group(1)
            new_lines.append(generate_placeholder(name, "function"))

        # Detect class
        class_match = re.match(r"class (.*?)", line.strip())
        if class_match:
            name = class_match.group(1)
            new_lines.append(generate_placeholder(name, "class"))

    with open(output_file, "w") as file:
        file.writelines(new_lines)
```



Observation:

◆ Before Tool

- Developers must manually write docstrings
- Time-consuming for large files
- Inconsistent documentation structure

◆ After Using Generator

- Functions and classes automatically detected
- Placeholder Google-style docstrings inserted
- Provides clear documentation template
- Speeds up development process
- Improves documentation consistency

◆ Limitations

- Does not perfectly handle indentation in all cases
- Does not analyze parameters automatically
- Designed for scaffolding, not full documentation